

IA Aplicada com LLMs

Aula 10 — O que MCP padroniza

A dor, e ela vem da Aula 05

```
@tool(schema=SCHEMA_POLITICA)
def consultar_politica(categoria: str) -> dict:
    ...
```

Cada ferramenta nova custou: **schema à mão**, validação de argumentos, classificação de erro entre recuperável e fatal, política de versionamento.

Aceitável quando a ferramenta é do próprio domínio.

Deixa de ser quando ela é **um sistema de terceiro** — e o mesmo cliente está sendo reescrito, com pequenas variações, por todo grupo que constrói um agente.

O problema em $M \times N$

SEM PROTOCOLO

M agentes

N sistemas

$M \times N$ integrações

COM PROTOCOLO

M agentes

N sistemas

$M + N$ implementações

O precedente mais próximo é conhecido de quem estuda Ciência da Computação:

Antes do **LSP**, cada editor precisava de uma integração para cada linguagem. O LSP definiu um protocolo entre editor e servidor de linguagem, e a matriz virou soma.

MCP faz a mesma operação sobre **agente** $\leftrightarrow$ **sistema**.

Onde a economia está — e onde não está

O protocolo **não escreve a integração**. Alguém ainda precisa implementar o servidor que fala com o sistema de tíquetes.

O que ele elimina é a MULTIPLICAÇÃO. Esse servidor é escrito **uma vez** e consumido por qualquer agente — inclusive por agentes que o autor não conhece.

Parte 1

Os três primitivos

A distinção não é de conteúdo. É de quem decide.

Primitivo	O que é	Quem decide invocar
Tool	operação executável, com efeito ou retorno	o modelo , no laço
Resource	dado legível, identificado por URI	a aplicação , ao montar o contexto
Prompt	modelo de interação parametrizado	o usuário , explicitamente

Apenas o primeiro é familiar: é o *tool calling* da Aula 03, com a declaração vindo de outro processo.

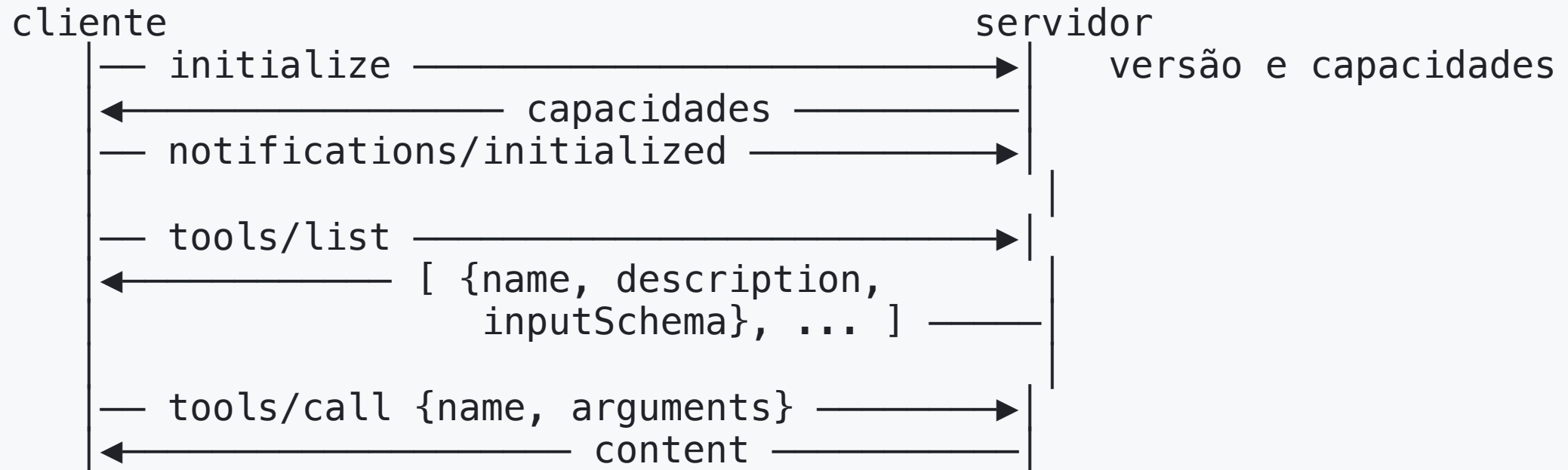
O resource é o primitivo novo mais importante

Uma ferramenta `ler_regulamento()` faria o mesmo trabalho. A diferença não é técnica — é **de controle**.

como FERRAMENTA	o modelo decide se lê -> pode não ler, ou ler quatro vezes
como RECURSO	a aplicação decide incluir -> a decisão é CÓDIGO: testável e barato

A regra prática: se a decisão de usar deve ser do modelo, é ferramenta; se deve ser do programa, é recurso.

A sessão MCP



JSON-RPC 2.0. O que volta de `tools/list` é a **declaração** — no mesmo formato que a Aula 03 usou.

Por que a negociação inicial existe

As duas pontas evoluem **separadamente**. Cliente e servidor declaram a versão do protocolo que falam e as capacidades opcionais que oferecem.

O que não for declarado ali não pode ser usado depois. É a mesma disciplina de contrato explícito que a Aula 03 estabeleceu para saída estruturada, aplicada à camada de transporte.

Os transportes

	stdio	Streamable HTTP
Como	processo filho, entrada e saída padrão	POST + SSE opcional
No laboratório	—	é o que a aula usa: servidor em terminal próprio
Rede	nenhuma	sim
Autenticação	isolamento do SO	OAuth 2.1 (desde 2025-03-26)
Quando	servidor local	servidor remoto

A escolha é de **topologia**, não de gosto. Servidor local que lê arquivos do disco do usuário **não deve escutar em porta alguma** — e a Aula 14 mostra o que acontece com quem inverte essa regra.

A especificação é móvel

Revisão	O que introduziu
2024-11	cliente-servidor; <i>tools, resources, prompts</i>
2025-03-26	Streamable HTTP; autorização OAuth 2.1
2025-06-18	saída estruturada; <i>elicitation</i>
2025-11-25	descoberta de autorização; <i>tasks</i> experimentais
2026-07-28	núcleo sem estado ; MCP Apps; política de depreciação

Cinco revisões em vinte meses.

Constrói-se sobre coisa que muda. A mesma frase valeu na Aula 02 para depreciação de modelo, e na Aula 09 para a API de um framework.

Parte 2

Consumir

O experimento decisivo

```
# ANTES – a declaração é dado local
declaracoes = [schema_de(f) for f in FERRAMENTAS.values()]

# DEPOIS – a declaração vem da sessão MCP
declaracoes = cliente.listar_ferramentas()           # tools/list
```

E o laço abaixo **não** muda nenhuma linha.

O que não mudou

Estado	não mudou	
laço	não mudou	
detector de laço	não mudou	<- continua sendo DA APLICAÇÃO
orçamento	não mudou	<- continua sendo DA APLICAÇÃO
motivo de término	não mudou	<- continua sendo DA APLICAÇÃO

Uma ferramenta local e uma ferramenta MCP **convivem na mesma lista** e são indistinguíveis para o modelo.

Três clientes, o mesmo servidor

01-sessao.py	JSON-RPC à mão, só json e urllib	síncrono
02-agente-com-mcp.py	cliente do SDK oficial	async
03-agente-langgraph-mcp	adaptador do framework	async

Cada degrau some com código — e o primeiro já cobra algo visível: o laço precisou virar `async`, e isso **não é do protocolo**.

Nenhum dos três inspeciona o que o servidor declara.

A afirmação que organiza a aula: MCP não é arquitetura de agente, é transporte de ferramenta

O protocolo **não** fornece: orçamento, detecção de laço, classificação de erro fatal, decisão de envolver um humano, ou *trace*.

Tudo o que torna um agente confiável continua sendo responsabilidade de quem escreve o agente.

Quem adota MCP esperando confiabilidade adotou a coisa certa pelo motivo errado.

O que fica

- $M \times N$ vira $M + N$ — e o precedente é o **LSP**
- Três primitivos, separados por **quem controla**
- A especificação **muda**, e a dependência é dívida
- O laço da Aula 05 **não mudou uma linha**
- Três clientes, do JSON-RPC cru ao adaptador — e o que cada um **esconde**

Próxima nota: publicar uma ferramenta — e a consequência de a descrição virar **prompt entregue a terceiros**.